PROJECT DEVELOPMENT PROGRESS REPORT

Okado Aaron Obbo	Reg No: **S24B23/015** | Student No: B29782	Last updated: **08 November 2026**

Project Title: Risk-Adaptive Zero Trust for Securing Hybrid Workforces Using Self-Supervised Anomaly Detection

Repository: github.com/asdcaaron/zta-ml-framework

Project Overview

This project designs and evaluates a Risk-Adaptive Zero Trust Architecture (ZTA) that integrates a dual-layer machine-learning detector: a supervised CNN-GRU for known-attack classification and a self-supervised autoencoder for zero-day anomaly detection. Both models feed a combined 0-1 risk score that the ZTA policy engine maps to ALLOW / MONITOR / CHALLENGE / BLOCK decisions. The system is compared against Legacy VPN and ZTA-Only baselines in a controlled four-way evaluation on the NSL-KDD dataset.

Component	Description
Preprocessing	NSL-KDD -> drop zero-variance -> log1p scaling -> MinMax -> 40 clean features (scaler.pkl, encoders.pkl)
CNN-GRU (Supervised)	Conv1D(64->128->256) + GRU(128->64), 327K params, 50 epochs. Known-attack classifier (Normal/DoS/P
Autoencoder (Self-Supervised)	Encoder 40->128->64->16 / Decoder 16->64->128->40. Trained on NORMAL only. Anomaly = recon. error
Flask API	/predict, /health endpoints on port 5000. Loads both models, computes combined risk score, enforces ZTA p
Zeek Pipeline	Docker-based: pcap -> Zeek conn.log -> Flask API -> audit.jsonl
Evaluation	4-way comparison: Legacy VPN vs ZTA Only vs ZTA+CNN-GRU vs ZTA+Autoencoder. Metrics: Accuracy,

Development Timeline

Each milestone below records what was done, the evidence (commits, files, artifacts), and key results. Pending milestones show the planned work ahead.

Milestone 1: Initial commit — ZTA + ML cybersecurity framework

DONE

Inception | 21 Aug 2026

Project repository initialised with the core framework structure. Defined the src/ module layout (preprocess, model, train, evaluate, api, zeek_capture, autoencoder) and the data/ directory for datasets, trained models, and results.

Evidence: git commit e27d1a8 | src/*.py | data/ | requirements.txt

Milestone 2: CNN-GRU trained + evaluation results + bug fixes

DONE

Model Training | 29 Aug 2026

Trained the supervised CNN-GRU classifier (Conv1D 64->128->256 + GRU 128->64, ~327K params) on the NSL-KDD dataset for 50 epochs. Achieved test accuracy 76% and validation accuracy 98.6%. Built the initial evaluation framework comparing the ML-augmented system against baselines.

Evidence: git commit 6165709 | data/best_model.keras | data/training_results.json

Results: Test acc 76% | Val acc 98.6% | 327K params

Milestone 3: Fixed evaluation baselines — ZTA+ML now correctly outperforms all systems

DONE

Evaluation | 29 Aug 2026

Corrected the baseline implementations so the four-way comparison was fair. After the fix, the ZTA+ML combination correctly outperforms Legacy VPN and ZTA-Only on all measured metrics.

Evidence: git commit d7537dd | src/evaluate.py

Milestone 4: Zeek-to-API pipeline integration completed

DONE

Pipeline Integration | 02 Sep 2026

Completed the full Zeek (Docker) capture pipeline: pcap files are processed by Zeek to produce conn.log, which is forwarded to the Flask API for real-time prediction. Every decision is logged to audit.jsonl.

Evidence: git commit 93ba9ff | src/zeek_capture.py | data/logs/conn.log | data/audit.jsonl

Milestone 5: Dataset noise fix — cleaner features for retraining

DONE

Data Quality | 02 Sep 2026

Removed noise from the NSL-KDD feature set. Dropped the zero-variance feature (num_outbound_cmds), applied log1p scaling on outlier-heavy columns, and reduced to 40 clean features. Saved the fitted scaler and encoders for consistent reuse.

Evidence: git commit 451c672 | src/preprocess.py | data/scaler.pkl | data/n_features.pkl

Results: 40 clean features | scaler.pkl | encoders.pkl

Milestone 6: Project retitled: Risk-Adaptive Zero Trust Using Self-Supervised Anomaly Detection

DONE

Project Direction | 15 Sep 2026

Refined the research focus to emphasise the self-supervised (label-free) autoencoder as the novel contribution, paired with the supervised CNN-GRU in a dual-layer risk-scoring architecture.

Evidence: git commit 77c6d9c

Milestone 7: Mid-semester milestone — summary PDF, demo dashboard, 40-feature pipeline fix

DONE

Mid-Semester | 13 Oct 2026

Produced the one-page mid-semester research summary PDF for UCU. Built a visual demo dashboard (HTML) and a PowerShell presentation script. Fixed a model/preprocess mismatch to ensure the full pipeline uses a consistent 40-feature space. GitHub repo at 7 commits on branch Aaron.

Evidence: git commits 2e9a84f, d763551, 5f59bfe | data/Midsemester_Summary_Okado_Aaron_Obbo.pdf | demo_dashboard.html | demo.ps1

Milestone 8: Self-supervised autoencoder trained for zero-day anomaly detection

DONE

Week 1 — Autoencoder Training | 06 Nov 2026

Trained the autoencoder (Encoder 40->128->64->16, Decoder 16->64->128->40) on NORMAL traffic only (67,343 samples) from NSL-KDD, using the existing saved scaler (transform, not refit) so it shares the exact 40-feature space as the CNN-GRU. Computed the 95th-percentile reconstruction-error threshold on normal test traffic (9,710 samples). Fixed the architecture to match the midsem spec. Created src/train_autoencoder.py. Pinned protobuf==6.31.1 to unblock TensorFlow.

Evidence: git commit 71198d8 | src/train_autoencoder.py | src/autoencoder.py | data/autoencoder_model.keras | data/ae_threshold.pkl | data/results/autoencoder_error_distribution.png | data/autoencoder_training_results.json

Results: Train loss 0.001935 | Val loss 0.000868 | Threshold 0.002682 | Recall on attacks 0.7643 | FPR on normal 0.0501 | Accuracy 0.8443 | F1 0.8482 | Mean error: normal 0.000792 vs attack 0.015131 (~19x)

Milestone 9: Full 4-way evaluation with real autoencoder predictions

DONE

Week 2 — Four-Way Evaluation | 06 Nov 2026

Updated src/evaluate.py so System D (ZTA+Autoencoder) uses real autoencoder predictions via load_autoencoder + the saved 95th-pct threshold, replacing the previous simulation fallback. Ran the full four-way evaluation on the same 22,543-sample test set with identical metrics across all systems. Regenerated all comparison artifacts. Key finding (honest deviation from expected narrative): the self-supervised autoencoder BEATS the CNN-GRU on accuracy and F1, while the CNN-GRU wins only on FPR. The autoencoder is also ~9x faster at inference. Its weakness is R2L detection (0.287) — those connections reconstruct like normal traffic.

Evidence: git commit a290aaf | src/evaluate.py | data/results/comparison_results.csv | data/results/summary.json | data/results/comparison_bar.png | data/results/confusion_matrices.png

Results: ZTA+Autoencoder: acc 0.8443, F1 0.8482, FPR 0.0501, 0.0656 ms/sample | ZTA+CNN-GRU: acc 0.7945, F1 0.7886, FPR 0.0452, 0.5889 ms/sample | ZTA Only: acc 0.7691 | Legacy VPN: acc 0.6664 | Autoencoder wins: accuracy, F1, DoS (0.911), Probe (0.882), U2R (0.687) | CNN-GRU wins: FPR (0.0452)

Milestone 10: GNS3 hybrid topology + attack traffic generation + end-to-end demo

DONE

Week 3 — GNS3 Lab | 14 Nov 2026

Designed and documented a GNS3 hybrid-workforce topology (4 segments: HQ 10.10.0.0/24, Branch 10.20.0.0/24, DMZ/Cloud 172.16.0.0/24, Internet attacker 203.0.113.0/24) with inside and outside attack sources, Zeek SPAN tap, and the Flask API as the ZTA policy enforcement point. Generated Scapy-crafted pcaps and Zeek-format conn.logs for all 5 NSL-KDD classes (Normal, DoS, Probe, R2L, U2R). Built the end-to-end demo orchestrator (run_gns3_demo.py) that scores traffic via three paths: conn.log path (reliable fallback), best-effort Zeek-Docker path, and a full-feature direct API path with windowed NSL-KDD profiles. Docker daemon was down so Zeek path is recorded as zeek_unavailable (honest). Results and audit records written to data/results/ and data/audit.jsonl.

Evidence: git commits 6848452, 42f77f4 | src/generate_attack_traffic.py | src/run_gns3_demo.py | data/gns3/zta_hybrid_topology.gns3 | data/gns3/topology.md | data/gns3/pcaps/*.pcap (5 files) | data/gns3/conn_logs/*_conn.log (5 files) | data/results/gns3_demo_results.json | data/audit.jsonl

Results: conn.log path: 122 flows scored (normal=24, dos=25, probe=35, r2l=20, u2r=18) | Full-feature path: DoS->BLOCK(0.930), R2L->BLOCK(0.899), U2R->CHALLENGE_MFA(0.572), Probe->ALLOW(0.028) | audit.jsonl: 7->165 lines (+158 records) | Scapy 2.8.0 installed; pcaps: normal=1KB, dos=2KB, probe=2.4KB, r2l=0.9KB, u2r=1.8KB | Zeek: Docker daemon DOWN (zeek_unavailable recorded)

Milestone 11: Threshold sensitivity sweep and zero-day recall on all attack classes

DONE

Week 4 — Robustness | 21 Nov 2026

Built src/robustness_analysis.py to evaluate the autoencoder's robustness. Swept the anomaly threshold from the 70th to the 99th percentile of normal reconstruction errors and recorded precision, recall, F1, and FPR at each point. Conducted a leave-one-class-out zero-day recall experiment: scored each NSL-KDD attack class independently against the saved 95th-pct threshold to measure how well the autoencoder detects each attack type purely from reconstruction error (no attack labels). Generated a threshold sweep chart, a per-class recall bar chart, and a JSON robustness report.

Evidence: src/robustness_analysis.py | data/results/threshold_sweep.png | data/results/zero_day_recall.png | data/results/robustness_report.json

Results: Separation ratio: 19.1x (attack mean error vs normal mean error) | Best F1=0.9214 at 81st pct threshold (precision=0.8717, recall=0.9770, fpr=0.190) | Saved 95th-pct threshold F1=0.8482 | Zero-day recall: DoS=0.9113 (22x sep), Probe=0.8819 (27.8x sep), U2R=0.6866 (24.1x sep), R2L=0.2873 (3.7x sep — R2L reconstructs like normal) | Test set: 22,543 samples (9,710 normal + 12,833 attack)

Milestone 12: Finalise results, write evaluation chapter, draft poster

DONE

Week 5 — Finalisation | 28 Nov 2026

Consolidated all results from Weeks 1–4. Wrote the full evaluation chapter as a formatted multi-page PDF (8 sections: introduction, dataset, system implementations, comparative results, threshold sensitivity, zero-day capability, discussion, conclusion). Drafted the A3-landscape research poster with a 3-column layout covering all key results, architecture pipeline, and zero-day recall figures. All documents match the project's existing PDF style (BLUE=#003087, GOLD=#C8A84B, Helvetica fonts, consistent table/border patterns).

Evidence: generate_evaluation_chapter.py | data/Evaluation_Chapter_Okado_Aaron_Obbo.pdf | generate_poster.py | data/Research_Poster_Okado_Aaron_Obbo.pdf

Results: Evaluation chapter: 8 sections, multi-page A4 PDF, all figures embedded | Research poster: A3 landscape, 3-column layout, 4-way comparison table, zero-day recall, threshold sensitivity | Progress PDF regenerated with Week 5 status = done

Milestone 13: Complete prototype, poster, submission; record demo

PENDING

Week 6 — Submission | Pending

Finalise the working prototype and research poster. Record a live demo video. Submit all deliverables.

Evidence: Not started

Current Status Summary

Metric	Value
Total milestones	13
Completed	12
In progress	0
Pending	1
Current phase	Week 5 complete — Week 6 (prototype completion, demo, submissio

Key Achievements So Far

- **Dual-layer ML pipeline complete:** Both the supervised CNN-GRU and the self-supervised autoencoder are trained and operational, sharing the same 40-feature preprocessing space.
- **Autoencoder (the novel contribution) is trained:** Achieves 0.8443 accuracy, 0.8482 F1, and 0.7643 recall on attacks with a label-free approach. Mean reconstruction error for attacks is ~19x higher than for normal traffic.
- **Full four-way evaluation done:** ZTA+Autoencoder beats all baselines on accuracy and F1, and is ~9x faster than the CNN-GRU at inference. Honest weakness: R2L detection (0.287) — those connections resemble normal traffic.
- **Robustness analysis complete (Week 4):** Threshold sweep (70th–99th pct) shows best F1=0.9214 at 81st pct. Zero-day recall: DoS=0.91, Probe=0.88, U2R=0.69. Confirmed weakness: R2L recall=0.29 (those connections look like normal traffic to the autoencoder — documented honestly).
- **Flask API operational:** /predict and /health endpoints serving both models with combined risk scoring and ZTA policy enforcement.

Next Steps

- **Week 5:** Finalise results, write the evaluation chapter, draft the research poster.
- **Week 6:** Complete the prototype and poster, record a live demo, and submit all deliverables.